

Informe Final: Electoral Integrity Analyzer

****Proyecto:**** Detección de anomalías en procesos electorales mediante análisis de datos

****Fecha:**** 3 de junio de 2026

1. Contexto

Este proyecto desarrolla una solución académica para identificar comportamientos inusuales en un proceso electoral simulado. El enfoque principal es la ciberseguridad y la integridad de los datos, analizando el ciclo electoral completo: desde el padrón de votantes hasta la publicación de resultados y la integridad de los logs del sistema.

****Objetivo:**** Aplicar técnicas de análisis de datos para detectar patrones sospechosos que puedan comprometer la integridad de una elección, utilizando únicamente datos sintéticos.

2. Metodología Detallada

2.1 Especificación de los Datasets

El sistema utiliza 9 datasets interrelacionados generados sintéticamente para simular un ecosistema electoral completo. Cada uno contiene datos específicos para el análisis:

1. ****01_padron_votantes.csv:**** Contiene el censo oficial. Campos clave: `voter\_id`, `fecha\_nacimiento`, `fecha\_defuncion`, `estado\_documento` (vigente/cancelado), `habilitado\_legalmente`.
2. ****02_asignacion_mesas.csv:**** Relaciona votantes con su ubicación autorizada. Campos: `voter\_id`, `mesa\_asignada`, `circunscripcion\_autorizada`.
3. ****03_registro_sufragio.csv:**** Registro de asistencia (check-in) en la jornada. Campos: `voter\_id`, `mesa\_voto`, `hora\_checkin`, `operador\_checkin`.
4. ****04_clasificacion_votos_manual.csv:**** Detalle de cada boleta procesada. Campos: `ballot\_id`, `clasificacion\_objetiva`, `clasificacion\_jurado`, `usuario\_clasificador`.
5. ****05_resultados_mesa.csv:**** Agregados finales. Campos: `electores\_habilitados`, `sufragantes\_registrados`, votos por candidato, `votos\_nulos`, `total\_reportado`.
6. ****06_logs_eventos.csv:**** Trazabilidad de acciones en el sistema. Campos: `usuario\_id`, `accion`, `rol\_usuario`, `hash\_antes`, `hash\_despues`.
7. ****07_integridad_archivos.csv:**** Control de hashes de actas y reportes. Campos: `archivo\_id`, `hash\_original`, `hash\_actual`, `estado\_integridad`.
8. ****08_alertas_esperadas.csv:**** "Ground Truth" para evaluar la precisión del sistema.
9. ****usuarios_sistema.csv:**** Roles y permisos de los operadores (JURADO, ADMIN, AUDITOR).

2.2 Técnica 1: Reglas Determinísticas (Reglas Simples)

Se aplican verificaciones lógicas de "cumplimiento obligatorio". Si una regla se rompe, la anomalía es inequívoca:

- * ****Elegibilidad (PAD):****
 - * `PAD-01`: Votante con `fecha\_defuncion` anterior a la elección pero con voto registrado.
 - * `PAD-02`: Votante con edad < 18 años habilitado para votar.
 - * `PAD-04`: Documentos marcados como "cancelados" o "inconsistentes" que ejercieron el voto.
- * ****Circunscripción (CIR/SUF):****
 - * `SUF-01`: Un mismo `voter\_id` con múltiples registros de sufragio (Doble voto).
 - * `CIR-01`: El `mesa\_voto` no coincide con el `mesa\_asignada` en el padrón.
- * ****Resultados (RES):****
 - * `RES-01`: Cantidad de sufragantes mayor a la de electores habilitados en una mesa.

```

*   `RES-03`: La suma de votos individuales (Candidatos + Nulos + Blancos)
no coincide con el `total_reportado`.
*   **Integridad y Logs (LOG/INT):**
*       `INT-01`: El `hash_actual` del archivo no coincide con el
`hash_original` (alteración de datos post-cierre).
*       `LOG-02`: Un usuario con rol "OPERADOR" intentando realizar acciones de
"ADMIN" (violación de RBAC).

```

2.3 Técnica 2: Análisis Estadístico (Z-score)

Se utiliza para detectar anomalías relativas al comportamiento del conjunto, donde no hay una regla fija pero el valor es altamente improbable.

```

**Fórmula aplicada:** $Z = (x - \mu) / \sigma$
*   Se marcan como anomalías los registros donde $|Z| \geq 3$ (Outliers
extremos).

```

****Variables analizadas y conclusiones:****

1. ****Participación por mesa (`participacion\_pct`):**** Un Z-score > 3 indica una participación extremadamente alta (cercana al 100% o superior a la media regional), lo que podría sugerir "relleno de urnas" (*ballot stuffing*).
2. ****Tasa de votos nulos/inválidos:**** Un Z-score muy alto indica que una mesa está invalidando votos de forma desproporcionada comparada con el resto del país, sugiriendo una posible supresión dirigida de votos.
3. ****Concentración del ganador:**** Si un candidato obtiene un porcentaje de votos que se aleja radicalmente de la distribución normal ($Z > 3$), se marca para revisión de coherencia política.
4. ****Carga de trabajo por operador:**** Identifica usuarios en los logs que realizaron una cantidad de acciones (modificaciones o registros) que se aleja de la media, lo cual podría indicar un operador comprometido o una automatización malintencionada.

2.4 Arquitectura de la Solución (Visión técnica)

La solución se implementó como un ****monolito modular por capas**** (orientado a demo académica), donde cada capa tiene una responsabilidad clara. Esta decisión reduce acoplamiento, facilita pruebas y permite justificar controles de ciberseguridad por componente (validación, auditoría, integridad, trazabilidad).

****Capas principales y módulos (evidencia en código):****

- ****Presentación / UI**:** `app.py` (Streamlit). Maneja navegación por etapas, estado de sesión y visualización. No contiene lógica de detección.
- ****Carga de datos**:** `src/data\_loader/csv\_loader.py` (lectura CSV) y `src/data\_loader/validators.py` (validaciones de esquema, tipos, valores permitidos y llaves referenciales).
- ****Dominio**:** `src/domain/alert\_types.py` (catálogo de alertas con severidad/acción) y `src/domain/schemas.py` (esquemas y valores permitidos).
- ****Detección**:** `src/detectors/\*\_detector.py` (reglas por etapa electoral). Cada detector expone `detect(datasets) -> DataFrame` y retorna alertas normalizadas.
- ****Orquestación (pipeline)**:** `src/pipeline/analysis\_pipeline.py` ejecuta detectores, consolida alertas, normaliza scores, calcula ranking y genera el resumen.
- ****Analítica**:** `src/analytics/statistics.py` (Z-score/outliers), `src/analytics/risk\_score.py` (score y ranking), `src/analytics/summary.py` (evaluación vs ground truth).
- ****Visualización**:** `src/visualizations/charts.py` construye gráficos Plotly (barras, histograma, boxplot).
- ****Evidencia/Reportes**:** `src/reports/report\_generator.py` exporta CSVs a `data/outputs/`.

****Flujo end-to-end (datos -> evidencia):****

1. Generación reproducible de datos sintéticos:
`scripts/generate\_synthetic\_data.py` (semilla fija `RANDOM\_SEED = 42`).
2. Carga y validación: `load\_datasets()` + `validate\_datasets()`. Si falla validación, se bloquea el análisis.

3. Ejecución del pipeline: ``AnalysisPipeline.run()`` invoca detectores por etapa (Strategy/Pipeline).
4. Consolidación y priorización: se normaliza severidad->score y se genera ranking por entidad (mesa/usuario/archivo/evento).
5. Exportación de evidencia: CSVs descargables para auditoría externa.

2.5 Controles de Ciberseguridad (Amenaza -> Control -> Evidencia)

La parte "cyber" no se limita a estadística; se implementan controles clásicos de seguridad operacional, auditoría e integridad, y se expresan como alertas con evidencia rastreable.

Amenaza / Riesgo	Control modelado	Alertas (ejemplos)	Evidencia en datasets	Evidencia en código
Violación de permisos (abuso de rol)	RBAC (matriz rol->acciones permitidas)	<code>`LOG-02`</code>	<code>`06_logs_eventos.csv`</code>	<code>`rol_usuario`, `accion`</code>
Falta de segregación de funciones	SoD (quien cambia no aprueba)	<code>`LOG-06`</code>	<code>`06_logs_eventos.csv`</code>	<code>`aprobado_por`, `usuario_id`</code>
Tampering post-cierre / cambios tardíos	Control de cambios post-cierre + aprobación	<code>`LOG-01`, `INT-04`</code>	<code>`06_logs_eventos.csv`</code>	(timestamp > cierre), <code>`07_integridad_archivos.csv`</code>
Fuerza bruta / intentos de acceso	Monitoreo de intentos fallidos	<code>`LOG-04`</code>	<code>`06_logs_eventos.csv`</code>	<code>`accion=login`, `resultado_accion=fallido`</code>
Alteración de evidencia digital	Integridad por hash (comparación original vs actual)	<code>`INT-01`, `LOG-05`</code>	<code>`06_logs_eventos.csv`</code>	<code>`hash_antes/hash_despues`</code>
Publicación inconsistente	Control de publicación vs firma (precondiciones)	<code>`INT-02`, `INT-03`</code>	<code>`07_integridad_archivos.csv`</code>	<code>`timestamp_firma/publicacion`, totales`</code>

Nota técnica: el proyecto modela **hashes** como evidencia de integridad. No implementa PKI real ni firma digital criptográfica; se usa el concepto para auditoría forense en un entorno académico/sintético.

2.6 Desarrollo Seguro (SSDF SP 800-218) aplicado al alcance del proyecto

No se afirma cumplimiento formal; se describe alineación práctica con decisiones visibles en el repo:

- **Separación por capas y mínimo acoplamiento**: UI separada de detectores/pipeline (reduce riesgo de cambios no controlados y facilita revisión).
- **Validación de entradas**: validación estructural y referencial antes de ejecutar análisis (``validators.py``).
- **Reproducibilidad y trazabilidad**: semilla fija + datasets sintéticos; permite auditoría repetible (misma entrada, mismo resultado).
- **Pruebas automatizadas**: ``pytest`` valida contrato de detectores, ejecución de pipeline y (ahora) que datasets generados pasan validación.

3. Resultados y Hallazgos

El sistema consolidó las alertas en un ranking de riesgo priorizado:

Ranking de Riesgo: Se utiliza una ponderación de severidad (Crítica=3, Alta=2, Media=1). Las mesas con un score acumulado > 12 se clasifican como de **"Revisión Prioritaria"**.

Evaluación contra Ground Truth (métrica aproximada): En una ejecución reproducible con ``RANDOM_SEED = 42``, la comparación por clave exacta ``(codigo_alerta, entidad_tipo, entidad_id)`` arrojó una **precision_aproximada ~ 0.67** y una **cobertura_aproximada ~ 0.66**. Estos valores son esperables

porque (1) el Z-score puede generar señales adicionales no etiquetadas en el ground truth sintético (falsos positivos deliberados para priorización) y (2) el ground truth se evalúa como conjunto de claves, no como verificación documental real.

4. Visualizaciones

1. ****Alertas por Etapa:**** Gráfico de barras que muestra que la mayoría de anomalías críticas ocurren en la etapa de ****Resultados**** e ****Integridad de Archivos****.
2. ****Histograma de Participación:**** Permite visualizar la "curva normal" de votación y las mesas que se encuentran en las "colas" de la distribución (outliers).
3. ****Boxplot de Nulos e Inválidos:**** Resalta visualmente las mesas que se salen del rango intercuartílico, facilitando la detección de supresión de votos.
4. ****Top 10 Mesas de Riesgo:**** Gráfico de barras horizontal que identifica las entidades exactas que requieren una auditoría forense inmediata.

5. Conclusiones

- Las ****reglas simples**** son efectivas para detectar errores administrativos y fraudes evidentes (fallecidos, dobles votos).
- El ****Z-score**** es indispensable para detectar fraudes más sutiles que se ocultan en volúmenes grandes de datos (supresión estadística o relleno de urnas).
- ****Limitación:**** El sistema es un indicador de anomalías, no una prueba de fraude. Toda alerta debe ser validada contra el acta física.
- ****Advertencia Ética:**** Este proyecto utiliza datos 100% sintéticos y no representa a ninguna institución o proceso electoral real.